Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 2 July 2014 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

NAME:

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following unilinear grammar G (axiom S):

$$G \begin{cases} S & \rightarrow & a\,T \\ T & \rightarrow & a \mid b\,T \mid a\,W \\ W & \rightarrow & \varepsilon \mid a\,T \end{cases}$$

Answer the following questions:

(a) By using the systematic method that produces a transition for each rule, transform grammar G into an automaton A, possibly nondeterministic.

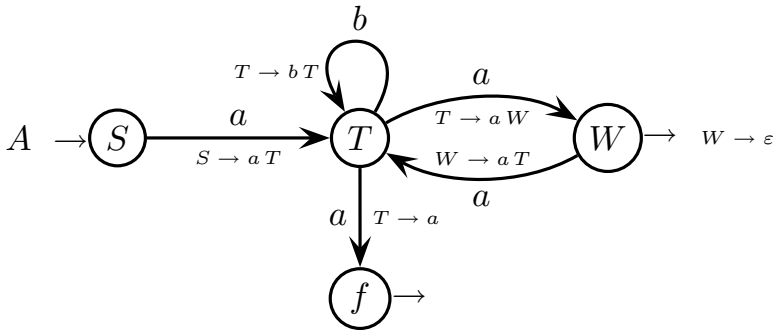
(b) If the automaton A found before is nondeterministic, construct an equivalent deterministic automaton A', by means of a systematic method of your choice.

(c) By using the method of the linear language equations, from grammar G obtain a regular expression R that generates language $L(G)$.

(d) (optional) By using the Berry-Sethi method, from the regular expression R found before obtain a deterministic automaton A'' that recognizes language $L(R)$.

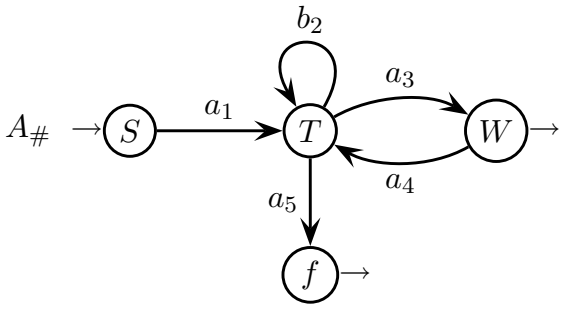
Solution

(a) Here is the transition automaton A of grammar G, with a transition per rule:

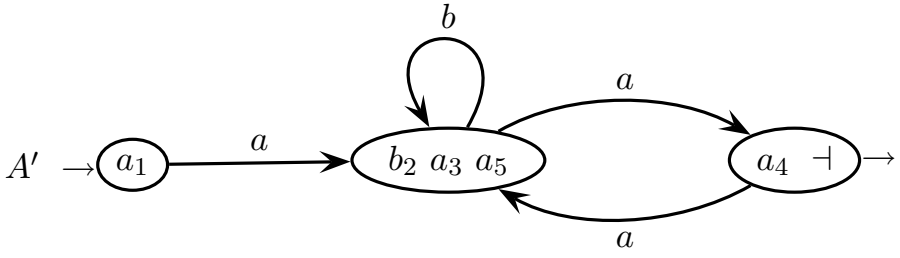


Automaton A is nondeterministic, since state T has two outgoing a-transitions.

(b) A way to determinize the automaton A obtained before is to use the Berry-Sethi algorithm, starting directly from A. Here it is:



initials	a_1
generators	*followers*
a_1	$b_2\ a_3\ a_5$
b_2	$b_2\ a_3\ a_5$
a_3	$a_4\ \dashv$
a_4	$b_2\ a_3\ a_5$
a_5	$\dashv$



Automaton A' is deterministic and also minimal, since the states a_1 and $b_2\,a_3\,a_5$ are distinguishable (the latter has a b-transition and the former does not).

Alternatively, one could determinize automaton A by means of the subset construction. If one did so, then the construction would simply group the states T and f of A, and it would yield a deterministic automaton identical to A'. This just happens by chance, as most frequently the Berry-Sethi algorithm and the subset construction yield different (though equivalent) automata.

(c) Here is the system of linear language equations of grammar G:

$$\begin{aligned} L_S &= \{a\}\ L_T \\ L_T &= \{a\} \cup \{b\}\ L_T \cup \{a\}\ L_W \\ L_W &= \{\varepsilon\} \cup \{a\}\ L_T \end{aligned}$$

Clearly, one ought to replace the third equation into the second one and so obtain an equation in only one unknown language:

$$\begin{aligned} L_S &= \{a\}\ L_T \\ L_T &= \{a\} \cup \{b\}\ L_T \cup \{a\}\ \big(\ \{\varepsilon\} \cup \{a\}\ L_T\ \big) \\ &= \{a\} \cup \{b\}\ L_T \cup \{a\} \cup \{a\,a\}\ L_T \\ &= \{a\} \cup \{a\,a,\ b\}\ L_T \\ L_W &= \{\varepsilon\} \cup \{a\}\ L_T \end{aligned}$$

By using the Arden identity, the second equation can be solved immediately:

$$L_T = \big(\, a\,a \ \mid\ b \,\big)^*\ a$$

and the other two equations as well, by substitution:

$$\begin{aligned} L_S &= a\ \big(\, a\,a \ \mid\ b \,\big)^*\ a \\ L_W &= \varepsilon \ \mid\ a\ \big(\, a\,a \ \mid\ b \,\big)^*\ a \end{aligned}$$

Therefore, the regular expression R is the following:

$$R = a\ \Big(\, a\,a \ \mid\ b \,\Big)^*\ a$$

It can also be transformed as follows, by using the well known identity of regular expressions $(\,\alpha \mid \beta\,)^* = \beta^*\,(\,\alpha\,\beta^*\,)^*$ and taking $\alpha = a\,a$, $\beta = b^*$:

$$a\ \Big(\ \underbrace{a\,a}_{\alpha}\ \mid\ \underbrace{b}_{\beta}\ \Big)^*\ a = a\ \underbrace{b^*}_{\beta^*}\ \Big(\ \underbrace{a\,a}_{\alpha}\ \underbrace{b^*}_{\beta^*}\ \Big)^*\ a = \big(\, a\ b^*\ a \,\big)^+$$

(d) Actually, the deterministic automaton A' obtained before is equivalent to the regular expression R. However, one can still obtain a deterministic automaton A'' directly from R, for instance by using again the Berry-Sethi algorithm. We just sketch how to do:

$$R_{\#} = a_1\ \Big(\, a_3\,a_4 \ \mid\ b_2 \,\Big)^*\ a_5\ \dashv$$

With this numbering, one immediately sees that the initials and the followers are the same as those in the construction of automaton A'. Therefore the automaton A'' will be identical to A'. We stop here with Berry-Sethi.

An alternative for constructing the automaton A'' is to use the Thompson method and thus obtain a nondeterministic automaton structurally identical to the regular expression R, and then to determinize it by cutting the spontaneous transitions and by using the subset construction or the Berry-Sethi algorithm.

2 Free Grammars and Pushdown Automata 20%

1. Consider the so-called *dangling else* case found in some programming languages. A way to solve it is to associate the dangling else branch to the *outermost* if conditional that can bear it. Here is an example, below on the left:

```
if expression then
    if expression then
        assignment
else
    assignment
```

example for question (a)

```
if expression then
    assignment;
    if expression then
        assignment;
        assignment
    else
        if expression then
            assignment
else
    assignment
```

example for question (b)

The dangling else branch is associated as shown by the alignment, that is to the outer if conditional. Notice this way of solving the dangling else case is quite different from the one usually adopted in most programming languages (e.g., the C language).

The branches of the if conditional contain only one statement: an assignment or a nested if conditional, and so on recursively. The expression and the assignment are generated by nonterminals E and A, resp., which do not need to be expanded.

Answer the following questions:

(a) Write a *BNF* grammar G, *not ambiguous*, that generates the if conditional (or simply an assignment) and solves the dangling else case as explained before.

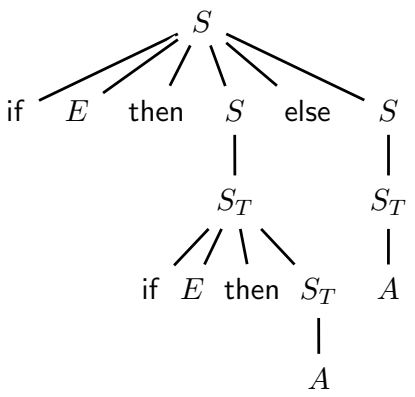
(b) (optional) Extend the language and let the branches of the if conditional contain a non-empty block of assignments and nested conditionals. Every statement in a block is followed by a semicolon ' ; ', except the last statement. See the example above on the right, where the last else is associated to the outermost if.

Write a *BNF* grammar G', *of any type*, that generates the extended language and solves the dangling else case as explained before. Generating a block of statements may require additional nonterminals, to be expanded.

Solution

(a) Here is grammar G (axiom S):

$$G \begin{cases} S & \rightarrow & \mathsf{if}\ E\ \mathsf{then}\ S\ \mathsf{else}\ S \\ S & \rightarrow & S_T \\ S_T & \rightarrow & \mathsf{if}\ E\ \mathsf{then}\ S_T \mid A \end{cases}$$

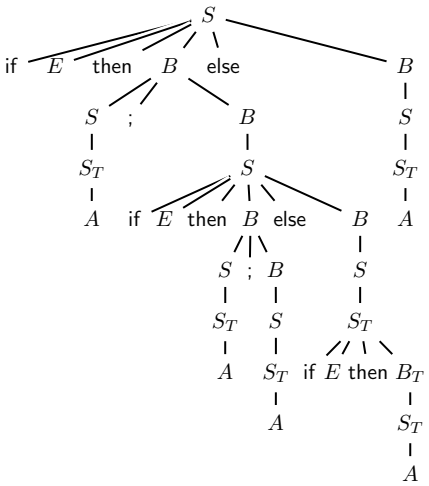


The underlying idea is that, once it is generated an if conditional that does not have an else branch, the inner conditionals may not have it either. The syntax tree on the right shows how grammar G works to generate example (a).

Grammar G is not ambiguous: the first rule is basically the standard Dyck one (think of "if E then" as an open parenthesis and of "else" as a closed one) and the third is basically right-linear, while the second is a pure categorization (which of course could be eliminated, though by making the grammar larger).

(b) Here is grammar G' (axiom S):

$$G' \begin{cases} S & \rightarrow & \mathsf{if}\ E\ \mathsf{then}\ B\ \mathsf{else}\ B \\ S & \rightarrow & S_T \\ S_T & \rightarrow & \mathsf{if}\ E\ \mathsf{then}\ B_T \mid A \\ \hline B & \rightarrow & S\ \text{`;'}\ B \mid S \\ B_T & \rightarrow & S_T\ \text{`;'}\ B_T \mid S_T \end{cases}$$



As before, the idea is that once it is generated an if conditional that does not have an else branch, the inner conditionals may not have it either. Thus the nonterminal that generates the block is split into two versions B and B_T, depending on whether it generates elements of type S or S_T, respectively, which may have an else branch or which surely do not have it.

Grammar G' is obtained from grammar G by adding the block rules to it and by forcing the conditional branches to contain the appropriate block type. Notice the language is extended ambiguously, since in a structure like "if E then if E then ass; ass" the last assignment may belong to the inner or outer if conditional. Thus grammar G' is ambiguous as well, but it solves the else case as prescribed.

2. One wishes one modeled a syntactic construct for defining a new data type: a *record with variant field*. Such a new record type looks like a standard record, e.g., a *struct* of the *C* language, which in addition may have one (or more) field(s) that can change name and type according to the value of a *discriminant* tag. Such a tag must be of enumeration type, declared separately.

   In the example below, the type dataDescriptor has two variants, according to the values of the discriminant tag kind of type enumeration addressType: in the variant with discriminant value absolute the record field is called absAddr and is a natural number; instead, in the variant with discriminant value offset the record field is called offAddr and is an integer.

   Consider the following restrictions:

   - the declarations of enumeration type must list at least two possible values (labels)
   - type declarations may be nested, meaning that the type of a record field may be specified by an identifier (declared somewhere else in the program) or by a full type declaration (possibly of record type) extensively written in place
   - although nested record declarations are permitted, it is forbidden to nest the case clauses; in other words, a variant field in a record may not in turn be of type record with variant field

   Here is an example, with an enumeration type and a type record with variant field:

```
type addressType = enum absolute, offset endEnum;
type dataDescriptor = record
    size: natural;
    case kind: addressType of
        if absolute then absAddr: natural;
        if offset then offAddr: integer;
    endCase;
endRecord
```

   The language consists of a non-empty list of declarations of type enumeration or record with variant field, as explained before.

   Write an *EBNF* grammar, not ambiguous, that generates the language fragment described above.

Solution

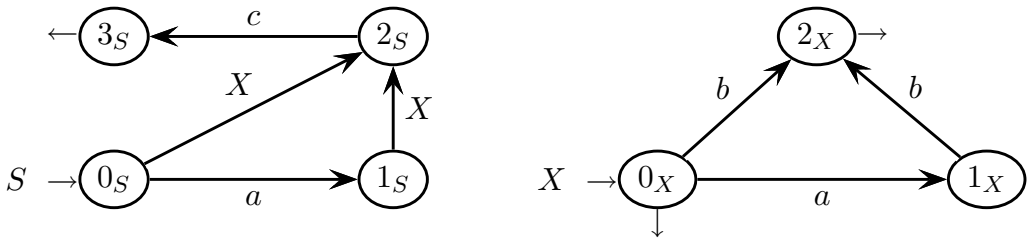
Here is an *EBNF* grammar that fulfills the specifications (axiom DECL_LST):

$$
\begin{cases}
\langle \text{DECL\_LST} \rangle \rightarrow \langle \text{TYP\_DECL} \rangle \ (\text{`;'} \ \langle \text{TYP\_DECL} \rangle \ )^* \\
\hline
\langle \text{TYP\_DECL} \rangle \rightarrow \text{type} \ \langle \text{ID} \rangle \ \text{`='} \ (\ \langle \text{ENUM\_TYP} \rangle \mid \langle \text{VREC\_TYP} \rangle \ ) \\
\hline
\langle \text{ENUM\_TYP} \rangle \rightarrow \text{enum} \ \langle \text{LABEL\_LST} \rangle \ \text{endEnum} \\
\langle \text{VREC\_TYP} \rangle \rightarrow \text{record} \ \langle \text{VFIELD\_LST} \rangle \ \text{endRecord} \\
\langle \text{REC\_TYP} \rangle \rightarrow \text{record} \ \langle \text{FIELD\_LST} \rangle \ \text{endRecord} \\
\langle \text{SCAL\_TYP} \rangle \rightarrow \text{natural} \mid \text{integer} \mid \langle \text{ENUM\_TYP} \rangle \\
\hline
\langle \text{LABEL\_LST} \rangle \rightarrow \langle \text{ID} \rangle \ \text{`,'} \ \langle \text{ID} \rangle \ (\text{`,'} \ \langle \text{ID} \rangle \ )^* \\
\langle \text{VFIELD\_LST} \rangle \rightarrow (\ \langle \text{VFIELD} \rangle \ \text{`;'})^+ \\
\langle \text{FIELD\_LST} \rangle \rightarrow (\ \langle \text{FIELD} \rangle \ \text{`;'})^+ \\
\hline
\langle \text{VFIELD} \rangle \rightarrow \langle \text{CASE\_FIELD} \rangle \mid \langle \text{ID} \rangle \ \text{`:'} \ (\ \langle \text{SCAL\_TYP} \rangle \mid \langle \text{VREC\_TYP} \rangle \ ) \\
\langle \text{FIELD} \rangle \rightarrow \langle \text{ID} \rangle \ \text{`:'} \ (\ \langle \text{SCAL\_TYP} \rangle \mid \langle \text{REC\_TYP} \rangle \ ) \\
\langle \text{CASE\_FIELD} \rangle \rightarrow \text{case} \ \langle \text{ID} \rangle \ \text{`:'} \ \langle \text{ID} \rangle \ \text{of} \ ( \ \langle \text{IF\_FIELD} \rangle \ \text{`;'})^+ \ \text{endCase} \\
\langle \text{IF\_FIELD} \rangle \rightarrow \text{if} \ \langle \text{ID} \rangle \ \text{then} \ \langle \text{FIELD} \rangle
\end{cases}
$$

Nonterminal ID should expand to a generic alphanumeric identifier, but here it is not shown. Notice that a variant field may not recursively contain variant fields. The example suggests that the last element of a field list has a terminator (identical to the separator), while the label lists and the whole language fragment do not do so; moreover, the enumerative lists ought to have at least two labels. The given solution faithfully models such a syntax, but these are tiny questionable details. The grammar is not ambiguous and reasonably correct. More compact solutions may exist.

3 Syntax Analysis and Parsing 20%

1. Consider the following recursive net of machines $\mathcal{M}$ (axiom S):



Answer the following questions:

(a) Draw the pilot of net $\mathcal{M}$, say if the net is of type $ELR(1)$, and if it is not, then list all the conflicts in the net.

(b) Net $\mathcal{M}$ *is not* of type $ELL(1)$. Draw the *PCFG* (*parser control-flow graph*) of net $\mathcal{M}$, show on it all the relevant guide sets and indicate where the net is not of type $ELL(1)$. Then try to find a new net $\mathcal{M}'$, equivalent to net $\mathcal{M}$, but which is of type $ELL(1)$.

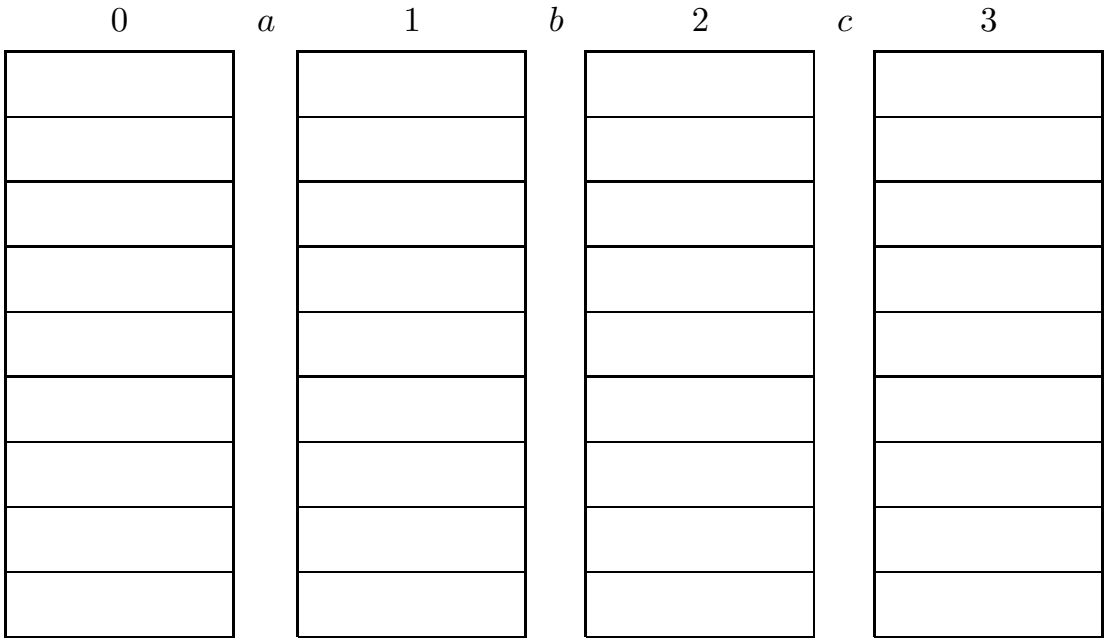
(c) (optional) By means of the Earley algorithm, analyze the string below:

$$a\,b\,c$$

which belongs to language $L(\mathcal{M})$. Draw the syntax tree (or all the trees) of this string. Please use the table prepared on the next page.

Earley vector of string $a\,b\,c$ (to be filled)

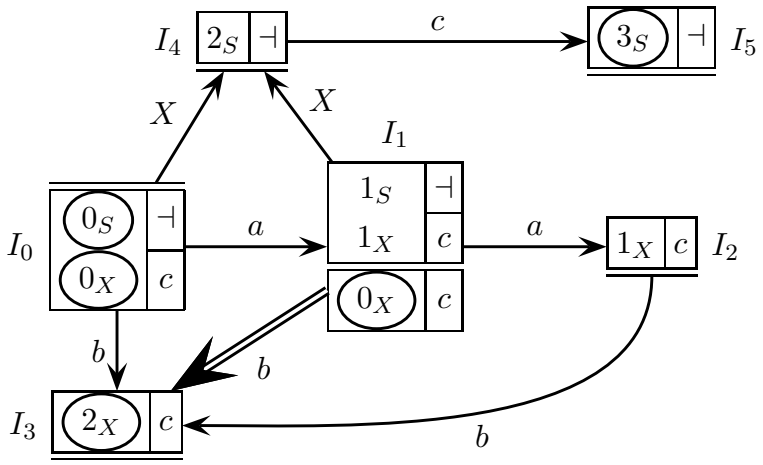
(the number of rows is not significant)



syntax tree(s) of string $a\,b\,c$ (to be drawn)

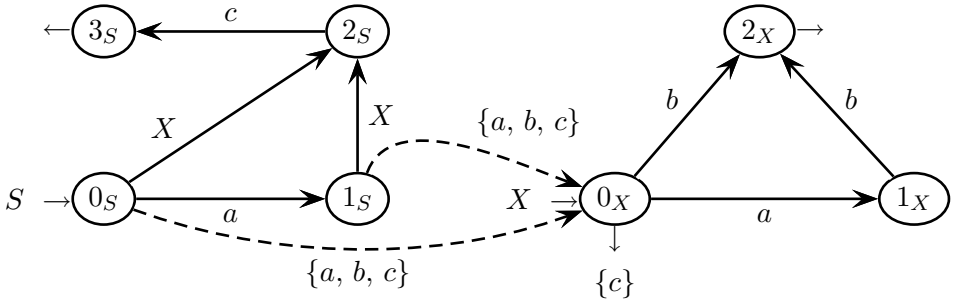
Solution

(a) Net $\mathcal{M}$ is not $ELR(1)$. Its pilot has six m-states and one convergence conflict, namely on the double convergent transition from I_1 to I_3 by b. Here it is:



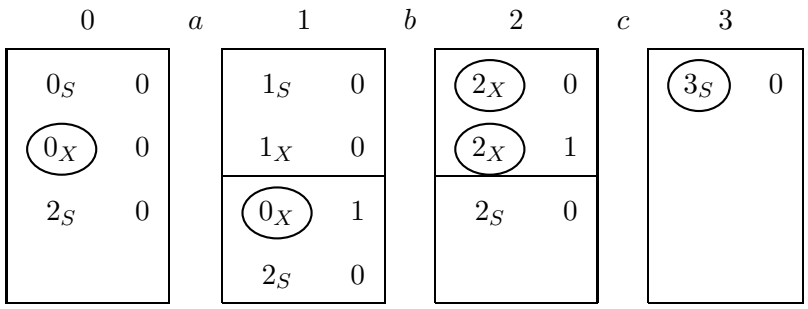
There are not any other conflicts.

(b) State 0_S of the $PCFG$ is a bifurcation point with non-disjoint look-ahead. The other bifurcation point is state 0_X, but here the look-ahead sets are disjoint.

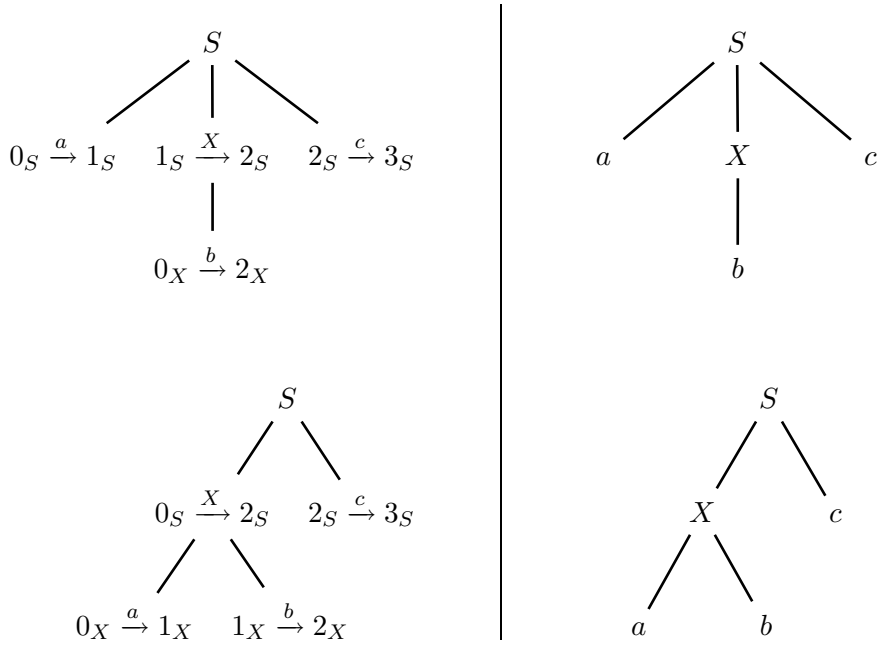


The language $L(\mathcal{M})$ is finite and any deterministic finite state automaton that recognizes L is immediately of type $LL(1)$, hence it is also of type $ELL(1)$.

(c) The Earley analysis is simple. Here is the Earley vector, with four states:



The string $a\,b\,c$ is ambiguous and has two syntax trees. Here they are:



4 Translation and Semantic Analysis 20%

1. In a hypothetical programming language, one wishes one transformed the iterative statements of the following type (cycle with final condition):

```
do
   a = a - 1;
end if (a = 0);
```

in three possible ways, shown by the three sample translations below:

(i)
```
repeat
   a = a - 1;
until (a = 0);
```

(ii)
```
repeat
   a = a - 1;
while ! (a = 0);
```

(iii)
```
while ! (a = 0)
   a = a - 1;
end while;
```

Notice the symbol '!' indicates the logical negation operator, as it usually happens in many programming languages. The body of the cycle is a generic block of statements, including possible nested iterative statements.

Answer the following questions:

(a) For each of the three ways shown above, say if the translation is syntactic and in such a case write its syntactic translation scheme (or its translation grammar), or say if it is not syntactic and explain why.

(b) (optional) If the translation is syntactic, say if it is computable by a finite state translator that scans the program one way, or possibly under what conditions such a finite state translation is computable.

Solution

(a) Translations (i) and (ii) are purely syntactic, while translation (iii) is not. In fact, the original iterative structure is generated by a source grammar rule of the following inclusive kind:

$I \rightarrow$ do B end if ‘ (’ E ‘) ’ ‘ ; ’

In this rule model, nonterminal I is the iterative structure itself, nonterminal B expands to a block of instructions, e.g., assignments, procedure invocations and inner structures of the same iterative type or of different types like conditionals, etc, and nonterminal E expands to a generic boolean or relational expression. The rules that expand nonterminals B and E are standard and not shown here.

Now, the corresponding target grammar rules for translations (i) and (ii) look like the following ones, respectively:

$I \rightarrow$ repeat B until ‘ (’ E ‘) ’ ‘ ; ’

and

$I \rightarrow$ repeat B while ‘ ! ’ ‘ (’ E ‘) ’ ‘ ; ’

and their relationship with the source rule is syntactic, since they have the same nonterminals ordered in the same way, and differ from the source rule only as for the terminals. Instead, a presumptive target grammar rule for translation (iii) should clearly look like this one:

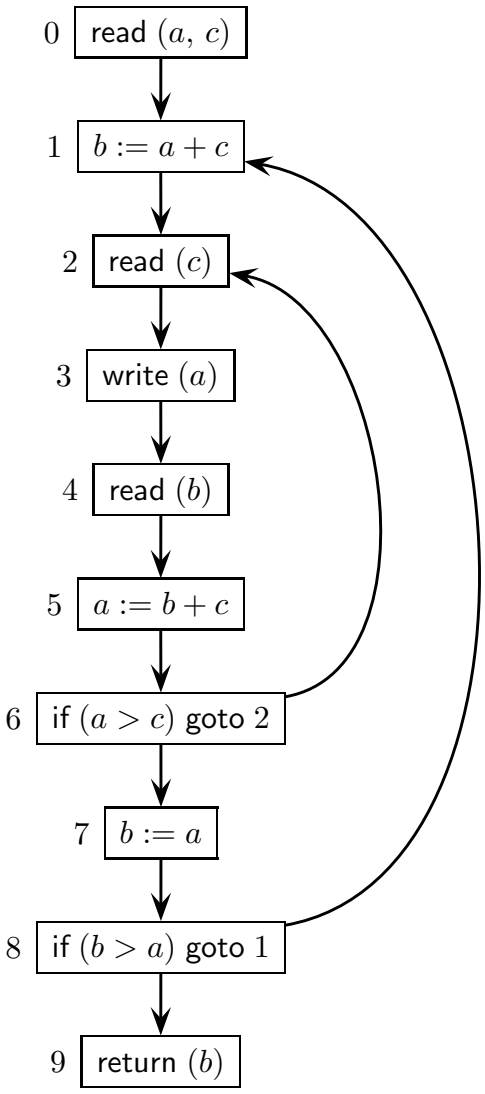
$I \rightarrow$ while ‘ ! ’ ‘ (’ E ‘) ’ B end while ‘ ; ’

Yet this last rule may not be associated with the source one to form a syntactic translation scheme (or grammar), as the ordering of nonterminals is different. Intuitively, in a purely syntactic way it is impossible to move the expression from the epilogue of the iterative structure to the prologue thereof, since the expression has an unbounded size and is a nested structure itself, at an arbitrary depth.

Of course, translation (iii) is computable in a semantic way, for instance by means of an attribute grammar. It is well known that an attribute grammar can swap the evaluation order of the subtrees of the syntax tree. In this case, it should simply swap the translations of the subtrees rooted at the child nodes B and E of the parent node I. Likewise, a one-sweep attribute grammar suffices.

(b) The question applies only to translations (i) and (ii). By inspecting the respective target rules shown before, it appears that for both (i) and (ii) it is only needed to replace the keyword do by the keyword repeat, and to replace the keyword digram end if by the one keyword until for (i) or by the digram keyword/operator while ! for (ii). Indeed, a finite translator could perform such replacements of lexical elements, which have a fixed size, but under a restrictive condition: that the whole source language should not use the keyword do or the keyword digram end if for any other purpose than delimiting the iterative structure type to be translated. In fact, a finite translator is unable to discriminate between different structural uses of the same lexical elements (while a syntactic translator can do).

2. Consider the following control-flow graph of a program (initial node 0) with three integer variables a, b and c, and an output parameter:



node	*use*
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

node	*def*
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Answer the following questions:

(a) Write a regular expression R (over the alphabet $[0-9]$) that represents the set of program execution traces, without considering the instruction semantics. Then say if such an expression R is representative also when considering the instruction semantics, and possibly refine it.

(b) By means of the flow equation method, find the *live variables* at the program nodes and write them on the control-flow graph by the node boxes. Please use also the tables prepared aside the program graph and those on the next page.

(c) (optional) Say which definitions of variable reach (*reaching definitions*) the program nodes number 9 (output node) and number 3, and explain why.

system of data-flow equations for live variables

node	*in equations*	*out equations*
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		

iterative solution table of the system of data-flow equations
(the number of columns is not significant)

#	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*
0										
1										
2										
3										
4										
5										
6										
7										
8										
9										

Solution

(a) Clearly the regular expression that generates all the program execution traces, irrespective of the instruction semantics, is the following:

$$R = 0\ \Big(\ 1\ \big(\ 2\ 3\ 4\ 5\ 6\ \big)^{+}\ 7\ 8\ \Big)^{+}\ 9$$

Semantically: obviously the condition at node 8 is always false, so that the outer cross iterates exactly once and the expression becomes the following one:

$$R' = 0\ 1\ \big(\ 2\ 3\ 4\ 5\ 6\ \big)^{+}\ 7\ 8\ 9$$

Clearly language $L\,(R')$ is a subset of language $L\,(R)$.

As for the condition at node 6, it depends on c, which is read inside of the loop $(2-6)$, and on a, which depends on b - also read inside of the loop $(2-6)$ - and again on c. Since the read operations are unpredictable, there is no semantic way to refine the condition. The rest of the program is a fixed sequence.

(b) Here are the definitions and uses of the three variables a, b and c:

node	*use*
0	–
1	a c
2	–
3	a
4	–
5	b c
6	a c
7	a
8	a b
9	b

node	*def*
0	a c
1	b
2	c
3	–
4	b
5	a
6	–
7	b
8	–
9	–

system of data-flow equations for live variables

node	*in equations*	*out equations*
0	$in\,(0) = out\,(0) - \{a,\, c\}$	$out\,(0) = in\,(1)$
1	$in\,(1) = \{a,\, c\} \cup \big(\ out\,(1) - \{b\}\ \big)$	$out\,(1) = in\,(2)$
2	$in\,(2) = out\,(2) - \{c\}$	$out\,(2) = in\,(3)$
3	$in\,(3) = \{a\} \cup out\,(3)$	$out\,(3) = in\,(4)$
4	$in\,(4) = out\,(4) - \{b\}$	$out\,(4) = in\,(5)$
5	$in\,(5) = \{b,\, c\} \cup \big(\ out\,(5) - \{a\}\ \big)$	$out\,(5) = in\,(6)$
6	$in\,(6) = \{a,\, c\} \cup out\,(6)$	$out\,(6) = in\,(7) \cup in\,(2)$
7	$in\,(7) = \{a\} \cup \big(\ out\,(7) - \{b\}\ \big)$	$out\,(7) = in\,(8)$
8	$in\,(8) = \{a,\, b\} \cup out\,(8)$	$out\,(8) = in\,(9) \cup in\,(1)$
9	$in\,(9) = \{b\} \cup out\,(9)$	$out\,(9) = \emptyset$

iterative solution table of the system of data-flow equations

#	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*
0	–	–	$a\,c$	–	$a\,c$	–	$a\,c$	–	$a\,c$
1	–	$a\,c$	–	$a\,c$	a	$a\,c$	a	$a\,c$	a
2	–	–	a	a	a	a	$a\,c$	a	$a\,c$
3	–	a	–	a	c	$a\,c$	c	$a\,c$	c
4	–	–	$b\,c$	c	$b\,c$	c	$b\,c$	c	$b\,c$
5	–	$b\,c$	$a\,c$	$b\,c$	$a\,c$	$b\,c$	$a\,c$	$b\,c$	$a\,c$
6	–	$a\,c$	a	$a\,c$	a	$a\,c$	$a\,c$	$a\,c$	$a\,c$
7	–	a	$a\,b$	a	$a\,b\,c$	$a\,c$	$a\,b\,c$	$a\,c$	$a\,b\,c$
8	–	$a\,b$	$a\,b\,c$	$a\,b\,c$	$a\,b\,c$	$a\,b\,c$	$a\,b\,c$	$a\,b\,c$	$a\,b\,c$
9	–	b	–	b	–	b	–	b	–

The solution converges in four steps. The rightmost column lists the variables that are alive at the node outputs (we omit to write them on the graph).

(c) It suffices to inspect the graph and look for the definitions of variables a, b and c that are closest to the program output node 9 and to the inner node 3. Therefore definitions a_5, b_7 and c_2 reach the entrance of the program output node 9, and no other definition reaches it. Instead, the entrance of node 3 is reached by definitions a_0, a_5 (through the backward arc from node 8 to node 1), b_1, b_4 (through the backward arc from node 6 to node 2) and c_2, and by no other definition. Notice that definition b_7 does not reach node 3, as it is suppressed by definition b_1. See the course textbook for more details.